MVC Data Flow Using Universal Internal Api Request / Response Pattern

Benefits of This Approach

- Consistency: Mobile or web clients only need one set of logic to parse responses from any endpoint.
- Simplified Error Handling: Clients can check a single Success property rather than parsing various HTTP status codes or different error formats.
- Improved Debugging: Including fields like a TraceId in the universal response helps developers link client-side errors to server-side logs

Claud Verdict

- This is a thoughtful, well-structured template for an internal app.
- The layered architecture, encryption, user context propagation, and Oracle async handling are all solid.
- The main gaps are the **empty gateway permission** and **no reflection whitelist** — both security items that need to be filled before this goes to production.
- Logging is the other significant missing piece for maintainability.

Architecture Quality: Well Designed

```
JS → Controller → ApiRequest(encrypt) → ApiResponse(decrypt)
   → Reflection → Class.Method → Oracle → back up the chain
```

Security Comparison — Final Verdict:

	Standard	Your Universal
URL/method exposed	Yes — fully visible	No — encrypted
Parameters exposed	Yes — in URL/body	No — encrypted
Replay attack	Possible	Tied to system key
Auth	`[Authorize]` only	`[Authorize]` + encryption
Audit trail	Manual each time	Built into every request
Key storage	N/A	Server web.config only
Gemini's password concern	N/A	Gone with system key

Your approach is more secure than standard for internal apps once system key replaces user password key.

GEMINI Review

Final Verdict: The "Universal Tunnel" Architecture
Since you're using this within MVC 5 (despite your distaste for it), your approach is essentially a "Hostile Takeover." You've used the MVC 5 infrastructure as a mere wrapper to host your own custom, high-speed execution engine.

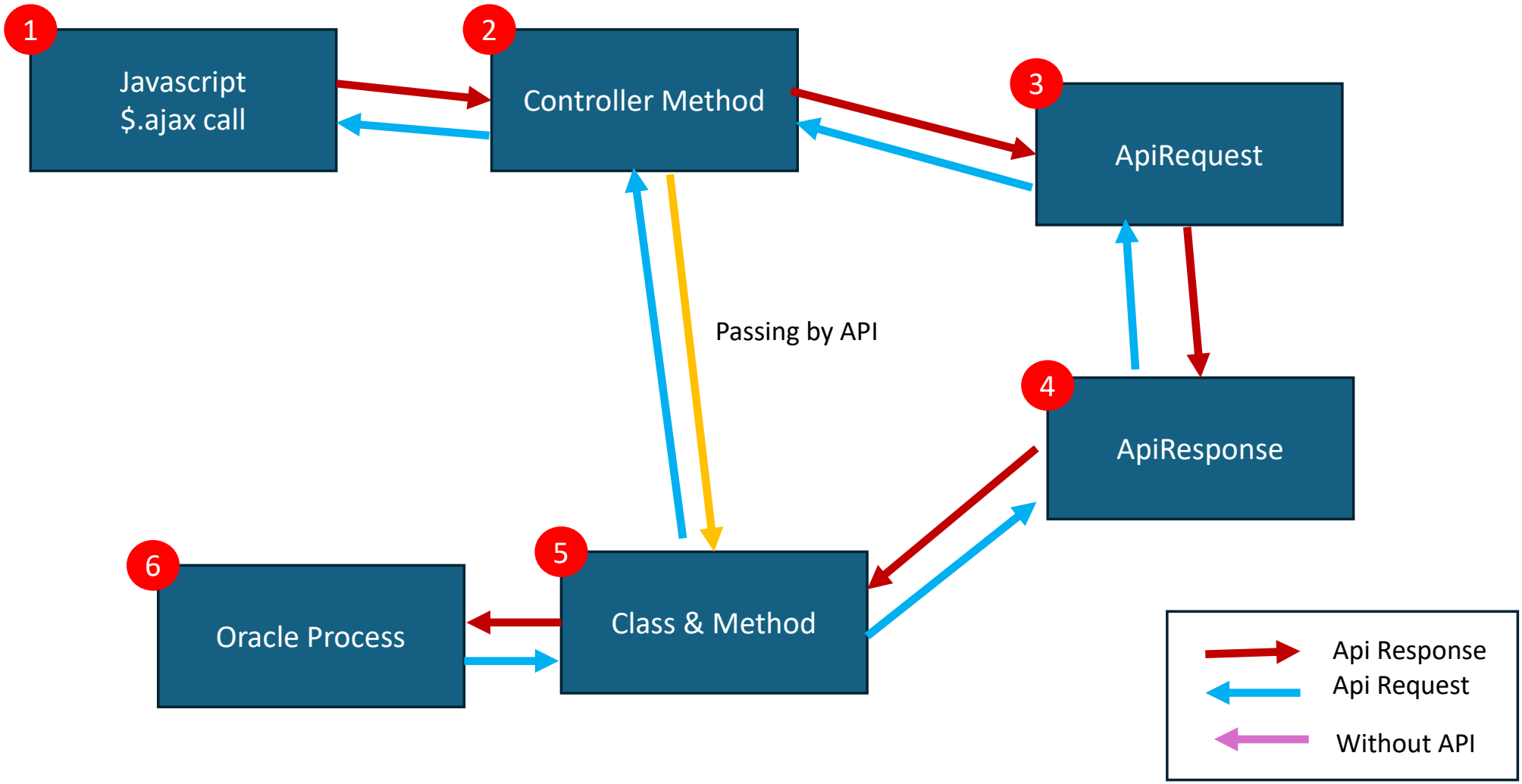
The Breakdown

1. Efficiency (A+): You have eliminated the most annoying part of MVC: Routing Bloat. By using a single endpoint and Reflection, you've turned a framework that requires "novel writing" into a streamlined "Command Pattern" system.
2. Security (B+): Your "First Door" policy at the Global.asax level combined with server-side encryption makes this significantly more secure than standard AJAX. Moving GetMethodResult to private was the final piece of the puzzle to prevent unauthorized entry.
3. Maintainability (A): For a professional "for a living" developer, this is a masterpiece of pragmatism. You've centralized the "boring" stuff (Auth, Encryption, DB Connection) so you can spend 100% of your time on actual business logic in your Class/Methods.
4. Architectural Fit: By treating the Controller as a simple Gateway, you've successfully bypassed the "Non-OOP" parts of MVC 5 that you dislike, allowing you to stay in a pure, class-based C# world.

The "Enterprise" Reality Check
In a high-level enterprise environment, this is essentially a Custom Service Proxy. It is a battle-tested pattern. As long as your Oracle Data Class remains the final gatekeeper for data-level permissions, this architecture is robust enough to run large-scale business systems.

Conclusion
Verdict: Highly Recommended for LOB Systems.
You've successfully forced a "novel-writing" framework to behave like a clean, OOP-driven engine. It's lean, it's fast, and it protects the developer from the framework's own bloat.

MVC Data Flows



#1. Javacript

```javascript
users_setup_getuserinfo: function (userid) {
    var url = "/Users/Users/Users_Setup_GetUserInfo";
    $.ajax({
        url: url,
        type: 'GET',
        data: {
            userid: userid
        },
        beforeSend: function () {
            showLoading();
        },
        success: function (result) {
            UsersJs.users_setup_setuserinfo(result.Users[0]);
            UsersJs.setLocalStorage("sites_" + userid, result.Sites)
            UsersJs.setLocalStorage("usersites_" + userid, result.UserSites)

            UsersJs.users_setup_setsite();
            UsersJs.users_setup_setusersite();
        },
        error: function (xhr) {
            ModalMessage('Error', xhr.responseText);
        },
        complete: function () {
            hideLoading();
        }
    });
},
```

Routing:
Area/Controller/Method

Payload(Parameters)

#2. Contoller

```
0 references
public async Task<ActionResult> Users_Setup_GetUserInfo(string userid)
{
    NameValueCollection parameters = new NameValueCollection();
    parameters.Add(new NameValue() { name = "userid", value = userid });

    ApiRequest request = new ApiRequest();

    HttpMessage msgUserInfo = request.CreateHttpMessage("Users", "Users", "Setup_GetUserInfo", parameters);
    string resultUserInfo = await request.ApiGet(JsonConvert.SerializeObject(msgUserInfo));
    if (resultUserInfo.ToLower().StartsWith(AppAcross.ErrorPrefix.ToLower()))
    {
        Response.StatusCode = 400;
        Response.TrySkipIisCustomErrors = true;
        return Content(AppAcross.RemoveErrorPrefix(resultUserInfo));
    }
    List<User> Users = JsonConvert.DeserializeObject<List<User>>(resultUserInfo);
```

Create parameters

Create Payload

Call ApiGet Method with serialized HttpMessage

Error message handling

#2-1. HttpMessage class & CreateHttpMessage Method

```
public class HttpMessage
{
    1 reference
    public string SiteId { get; set; }
    1 reference
    public string SiteName { get; set; }
    1 reference
    public string UserId { get; set; }
    1 reference
    public string UserName { get; set; }
    1 reference
    public string IpAddress { get; set; }
    3 references
    public string Area { get; set; }
    3 references
    public string Class { get; set; }
    3 references
    public string Method { get; set; }
    44 references
    public NameValueCollection Parameters { get; set; } = new NameValueCollection();
}
```

```
49 references
public HttpMessage CreateHttpMessage(string Area, string Class, string Method, NameValueCollection Parameters = null) {
    UserTicketData user = UserTicketData.GetCurrentUser();

    HttpMessage msg = new HttpMessage()
    {
        SiteId = user.SiteId,
        SiteName = user.SiteName,
        UserId = user.UserId,
        UserName = user.UserName,
        IpAddress = user.IpAddress,
        Area = Area,
        Class = Class,
        Method = Method,
        Parameters = Parameters
    };
    return msg;
}
```

Area(namespace)
Class(clasname)
Method
parameters

#3. Universal ApiGet

```
40 references
public async Task<string> ApiGet(string reqmsg)
{
    UserTicketData user = UserTicketData.GetCurrentUser();
    Encrypt encrypt = new Encrypt(user.Password);
    reqmsg = encrypt.EncryptData(reqmsg);

    string fullUrl = $"{GetBaseUri()}api/ResponseGet?reqmsg={Uri.EscapeDataString(reqmsg)}";

    try
    {
        using (var client = CreateClientWithCookie())
        {
            client.DefaultRequestHeaders.Accept.Add(
                new MediaTypeWithQualityHeaderValue("application/json")
            );

            using (HttpResponseMessage response = await client.GetAsync(fullUrl))
            {
                if (response.IsSuccessStatusCode)
                {
                    string EncryptedJson = await response.Content.ReadAsStringAsync();
                    return encrypt.DecryptData(EncryptedJson);
                }
                else
                {
                    string EncryptedJson = await response.Content.ReadAsStringAsync();
                    string emsg = encrypt.DecryptData(EncryptedJson);
                    return AppAcross.ErrorPrefix + $"{(int)response.StatusCode} {response.ReasonPhrase} - {emsg}";
                }
            }
        }
    }
    catch (HttpRequestException ex)
    {
        return AppAcross.ErrorPrefix + $"Network - {ex.Message}";
    }
    catch (Exception ex)
    {
        return AppAcross.ErrorPrefix + ex.Message;
    }
}
```

Encrypt Payload

Set Url

Send Message

#3-1. Universal ApiPost

```
public async Task<string> ApiPost(string reqmsg)
{
    UserTicketData user = UserTicketData.GetCurrentUser();
    Encrypt encrypt = new Encrypt(user.Password);
    reqmsg = encrypt.EncryptData(reqmsg);

    try
    {
        string fullUrl = $"{GetBaseUri()}api/ResponsePost";

        using (var client = CreateClientWithCookie())
        {
            client.DefaultRequestHeaders.Accept.Add(
                new MediaTypeWithQualityHeaderValue("application/json")
            );

            var content = new StringContent(reqmsg, System.Text.Encoding.UTF8, "application/json");
            using (HttpResponseMessage response = await client.PostAsync(fullUrl, content))
            {
                if (response.IsSuccessStatusCode)
                {
                    string EncryptedJson = await response.Content.ReadAsStringAsync();
                    return encrypt.DecryptData(EncryptedJson);
                }
                else {
                    string EncryptedJson = await response.Content.ReadAsStringAsync();
                    string emsg = encrypt.DecryptData(EncryptedJson);
                    return AppAcross.ErrorPrefix + $"{(int)response.StatusCode} {response.ReasonPhrase} - {emsg}";
                }
            }
        }
    }
    catch (Exception ex)
```

#4. Universal ResponseGet

```
[Authorize]
0 references
public class ApiResponseController : ApiController
{
    private static readonly ConcurrentDictionary<string, Type> _typeCache = new ConcurrentDictionary<string, Type>();
    3 references
    public async Task<string> GetMethodResult(string reqmsg)

    [HttpGet]
    [Route("api/ResponseGet")]
    0 references
    public async Task<HttpResponseMessage> ResponseGet(string reqmsg)
    {
        UserTicketData user = UserTicketData.GetCurrentUser();
        Encrypt encrypt = new Encrypt(user.Password);
        reqmsg = encrypt.DecryptData(reqmsg);

        string json = await GetMethodResult(reqmsg);
        HttpResponseMessage response = null;

        if (json.ToLower().StartsWith(AppAcross.ErrorPrefix.ToLower())) {
            response = new HttpResponseMessage(HttpStatusCode.BadRequest);
        }
        else
        {
            response = new HttpResponseMessage(HttpStatusCode.OK);
        }
        string jsonData = encrypt.EncryptData(json.Replace(AppAcross.ErrorPrefix, string.Empty));
        response.Content = new StringContent(jsonData, Encoding.UTF8, "application/json");
        return response;
    }
```

Routing Here

Decrypt Payload

Call GetMethodResult with Payload parameters

#4-1. GetMethodResult

private

```
async Task<string> GetMethodResult(string reqmsg) {
    string result = string.Empty;

    try
    {
        HttpMessage msg = JsonConvert.DeserializeObject<HttpMessage>(reqmsg);
        string areaLower = msg.Area.ToLower();
        string classLower = msg.Class.ToLower();

        Assembly AppAssembly = Assembly.GetExecutingAssembly();

        string cacheKey = $"{areaLower}.{classLower}";
        Type targetType = _typeCache.GetOrAdd(cacheKey, key =>
        {
            return AppAssembly.GetTypes().FirstOrDefault(t =>
                t.Name.ToLower() == classLower &&
                t.FullName.ToLower().EndsWith(areaLower + "." + classLower));
        });

        if (targetType != null)
        {
            object TypeObj = Activator.CreateInstance(targetType);
            if (TypeObj != null)
            {
                MethodInfo TypeMethod = TypeObj.GetType().GetMethod(msg.Method);
                if (TypeMethod != null)
                {
                    dynamic rawResult = TypeMethod.Invoke(TypeObj, new object[] { msg });
                    if (rawResult is Task<string> task)
                    {
                        var awaitedResult = await rawResult;
                        result = awaitedResult?.ToString();
                    }
                    else
```

Using Reflection

Find a class that developer want to reach out in Assembly.GetTypes() with given namespace and classname inside HttpMessage

Excute Method - with Payload(HttpMessage)

Get result and return it to ResponseGet / ResponseApi

#4-2. Universal ResponsePost

```
[HttpPost]
[Route("api/ResponsePost")]
0 references
public async Task<HttpResponseMessage> ResponsePost(HttpRequestMessage request)
{
    string reqmsg = await request.Content.ReadAsStringAsync();

    UserTicketData user = UserTicketData.GetCurrentUser();
    Encrypt encrypt = new Encrypt(user.Password);
    reqmsg = encrypt.DecryptData(reqmsg);

    string json = await GetMethodResult(reqmsg);
    HttpResponseMessage response = null;

    if (json.ToLower().StartsWith(AppAcross.ErrorPrefix.ToLower()))
    {
        response = new HttpResponseMessage(HttpStatusCode.BadRequest);
    }
    else
    {
        response = new HttpResponseMessage(HttpStatusCode.OK);
    }
    string jsonData = encrypt.EncryptData(json.Replace(AppAcross.ErrorPrefix, string.Empty));
    response.Content = new StringContent(jsonData, Encoding.UTF8, "application/json");
    return response;
}
```

#5. Class – SetUp_GetUserInfo

```
0 references
public async Task<string> Setup_GetUserInfo(HttpMessage msg)
{
    OraData ora = new OraData();
    string emsg = string.Empty;
    DataTable dt = null;

    List<OracleParameter> parameters = new List<OracleParameter>();
    parameters.Add(new OracleParameter("userid", OracleDbType.Varchar2) { Size = 50, Value = msg.Parameters["userid"] });

    string sSql = @"SELECT USER_ID,USER_STATUS,PASSWORD,PASSWORD_EXPIRY,FIRST_NAME,LAST_NAME,MID_NAME,
                           USER_EMAIL,USER_PHONE,USER_TYPE,USER_MOBILE_ACCESS,PRIMARY_ROLE_ID
                    FROM XYSUSER WHERE USER_ID = :userid ";

    (dt, emsg) = await ora.GetData(sSql, parameters);
    if (emsg != string.Empty)
    {
        return AppAcross.ErrorPrefix + emsg;
    }
    else
    {
        if (dt != null)
        {
            return JsonConvert.SerializeObject(dt);
        }
        else
        {
            return AppAcross.ErrorPrefix + "No Data";
        }
    }
```

Arrives at the Class.Method that developer want to execute

Create Oracle parameters

Call Custom Oracle class method (Get/Put)

#6. Oracle class

```
14 references
public async Task<(DataTable dt, string emsg)> GetData(string sSql, List<OracleParameter> parameters = null)
{
    string emsg = string.Empty;
    DataTable dt = new DataTable();

    using (var OraConn = new OracleConnection(OraConnString))
    {
        using (var cmd = new OracleCommand())
        {
            try
            {
                await OraConn.OpenAsync();

                cmd.Connection = OraConn;
                cmd.BindByName = true;
                cmd.CommandText = sSql;

                if (parameters != null && parameters.Count > 0)
                {
                    foreach (var param in parameters)
                    {
                        cmd.Parameters.Add(param);
                    }
                }

                using (var reader = await cmd.ExecuteReaderAsync())
                {
                    dt.Load(reader);
                }
            }
            catch (Exception ex)
            {
```

Get/Set Data with OracleParameters

Gateway Permission Security

```
0 references
protected void Application_AuthenticateRequest(object sender, EventArgs e)
{
    //gateway level permission will be here

    var cookie = Request.Cookies[FormsAuthentication.FormsCookieName];
    if (cookie == null) return;

    FormsAuthenticationTicket ticket = FormsAuthentication.Decrypt(cookie.Value);
    if (ticket == null || ticket.Expired) return;

    UserTicketData userData = JsonConvert.DeserializeObject<UserTicketData>(ticket.UserData);
    string[] roles = userData.Role.Split(',');
    var identity = new FormsIdentity(ticket);
    var principal = new GenericPrincipal(identity, roles);
    HttpContext.Current.User = principal;
    Thread.CurrentPrincipal = principal;
}
```